

Proof of Concept Sensorsysteem

Thijs Buisman 1855662 HU

1. Aanleiding en keuze voor de lasersensor

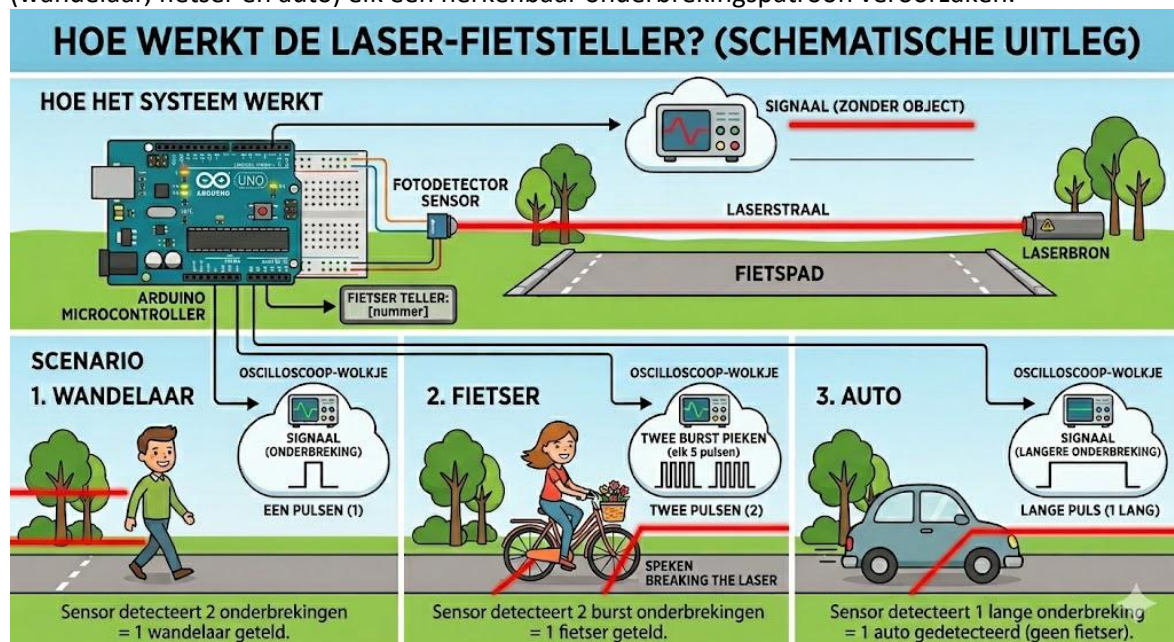
Binnen het onderzoek naar het gebruik van de buitenruimte in de wijk Cartesius was er behoefte aan een methode om passanten continu en objectief te tellen. De handmatige observatiemethode SOPARC levert waardevolle data op, maar is arbeidsintensief en beperkt tot momentopnames. Daarom is onderzocht of sensortechnologie dit proces kon automatiseren.

Bij de keuze voor een geschikte meettechniek zijn meerdere alternatieven overwogen:

- Camera met computer vision: betrouwbaar en nauwkeurig, maar maakt beeldopnames van mensen. Dit stuit op privacybezwaren en is niet AVG-proof voor gebruik in de openbare ruimte.
- PIR-sensor (bewegingssensor): goedkoop en eenvoudig, maar te ongericht. Reageert op warmte en beweging zonder onderscheid te maken tussen verschillende gebruikerstypen.
- Ultrazone sensor (HC-SR04): geschikt voor afstandsmeting, maar minder betrouwbaar bij smalle doorgangen met meerdere passanten tegelijk.
- Lasersensor met fotodiode: nauwkeurig, AVG-proof (geen beeldopname), kostenefficiënt en geschikt voor het detecteren van patronen per gebruikerstype.

De keuze is gevallen op het lasersysteem. Dit systeem werkt volledig anoniem er worden geen beelden gemaakt en geen persoonsgegevens opgeslagen wat aansluit bij de privacy-eisen die gelden voor onderzoek in een woonwijk.

De werking van het systeem is schematisch weergegeven in Figuur 1. Hierin is te zien hoe de laserstraal tussen de laserbron en de fotodetector loopt, en hoe verschillende gebruikerstypen (wandelaar, fietser en auto) elk een herkenbaar onderbrekingspatroon veroorzaken.



Figuur 1 - Schematische weergave van de werking van de laser-fietsteller (Bron: nano banana 2)

2. Systeemeisen

Om te kunnen beoordelen of het prototype voldoet aan de gestelde verwachtingen, zijn vooraf systeemeisen opgesteld. Deze eisen zijn geprioriteerd volgens de MoSCoW-methode zo als te zien is in Tabel 1.

Tabel 1 – Eisenlijst sensorsysteem

MoSCoW	Eis
Must have	Het systeem detecteert een onderbreking van de laserstraal betrouwbaar bij minimaal 2 meter afstand tussen laser en fotodiode.
Must have	Elke detectie wordt opgeslagen met een correcte tijdstempel (datum en tijd) op de SD-kaart.
Must have	Het systeem onderscheidt minimaal drie gebruikerstypen: voetganger, fietser en auto.
Must have	Het systeem werkt minimaal 8 uur aaneengesloten op een powerbank van 5000 mAh.
Should have	Het systeem werkt betrouwbaar bij direct zonlicht zonder valse detecties.
Should have	De foutmarge ten opzichte van handmatige telling bedraagt maximaal 15%.
Should have	Het systeem logt geen dubbeltellingen bij één passage.
Could have	Het systeem detecteert de looprichting van een passant.
Could have	De data wordt automatisch geëxporteerd naar een leesbaar CSV-bestand met kolomkoppen.
Won't have	Draadloze datatransmissie (WiFi/LoRa).
Won't have	Realtime monitoring op afstand.
Won't have	Automatische classificatie via machine learning.

3. Componentenkeuze en kostenoverzicht

Op basis van de gekozen meettechniek zijn de benodigde componenten geselecteerd. Daarbij is gelet op beschikbaarheid, prijs en geschiktheid voor buiteninzet. Bewust is gekozen voor een opstelling zonder draadloze verbinding, zodat het systeem volledig zelfstandig functioneert en niet afhankelijk is van een netwerkverbinding. Onderstaande Tabel 2 toont een overzicht van alle onderdelen en de bijbehorende kosten.

Tabel 2 - Kostenoverzicht ontwikkeling prototype

Onderdeel	Specificatie	Kosten
Arduino Nano	Microcontroller voor de verwerking	€ 9,99
Laser module KY-008	Laserbron voor de straalonderbreking	€ 1,50
Clock module DS3231	2 stuks (voor exacte tijdsregistratie)	€ 5,40
Mini SD module	SPI interface voor data-opslag	€ 2,27
Micro-SD Kaart 32GB	Lokale data-opslag	€ 17,99
Photodiode SGPD5051C6	5mm flat lens (detectie van de straal)	€ 1,85
Weerstand kit	Diverse componenten voor circuit	€ 2,50
Kabel rood/zwart	Solid core 0,25 mm (25m)	€ 0,79
Solder board	2 stuks voor montage	€ 1,40
3D print filament	150 gram (voor behuizing)	€ 2,25
Powerbank	2 stuks (5000 mAh) voor voeding	€ 37,98
Totaal		€ 83,92

Opmerking: verzendkosten zijn niet meegenomen omdat een deel van de componenten al in eigen bezit was.

4. Aansluiting en opbouw van het circuit

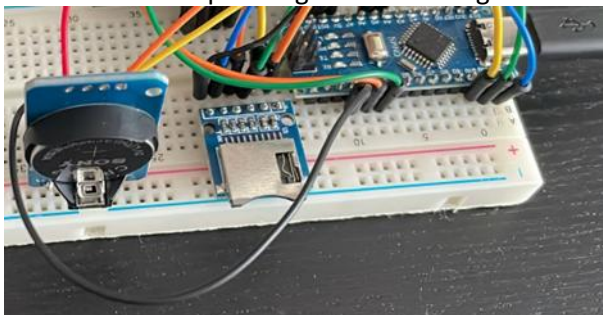
4.1 Test op breadboard

Voordat de componenten definitief werden gesoldeerd, is de volledige schakeling eerst opgebouwd op een breadboard. Dit maakt het eenvoudig om aansluitingen te testen en fouten te corrigeren zonder schade aan de onderdelen. Tijdens de breadboard-fase is gecontroleerd of:

- De fotodiode het lasersignaal correct ontvangt en verwerkt.
- De RTC-module de juiste tijd doorgeeft aan de Arduino.
- De SD-kaartmodule data wegschrijft zonder fouten.
- De Arduino Nano de signalen correct interpreteert en logt.

Tijdens deze fase is ook een belangrijk probleem ontdekt met de aansluiting van de fotodiode. Eerst was de fotodiode aangesloten op digitale pin D2 met een 1M Ω pull-down weerstand naar GND. Binnen werkte dit correct, maar buiten in direct zonlicht zag het systeem continu een laser terwijl die er niet was. De hoge weerstandswaarde maakte de fotodiode te gevoelig voor omgevingslicht. Dit is opgelost door over te stappen op een 10k Ω pull-down weerstand en de fotodiode aan te sluiten op analoge pin A3. Hierdoor kon in de code een drempelwaarde worden ingesteld waarboven een signaal pas als laser wordt gedetecteerd.

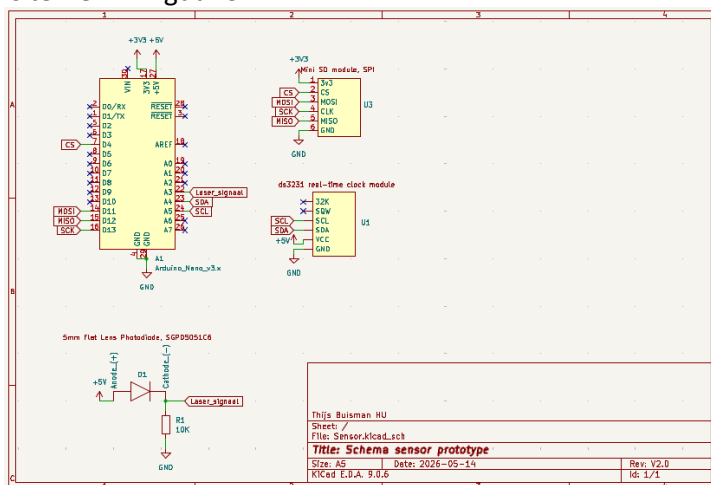
De breadboard-opstelling is te zien in Figuur 2.



Figuur 2 – Test sensor schakling op breadbord.

4.2 Schematisch ontwerp in KiCad

Het aansluitschema is uitgewerkt in KiCad. Dit schema toont hoe de Arduino Nano, de RTC-module (DS3231), de SD-kaartmodule en de fotodiode op elkaar zijn aangesloten. De schematische weergave is te zien in Figuur 3.



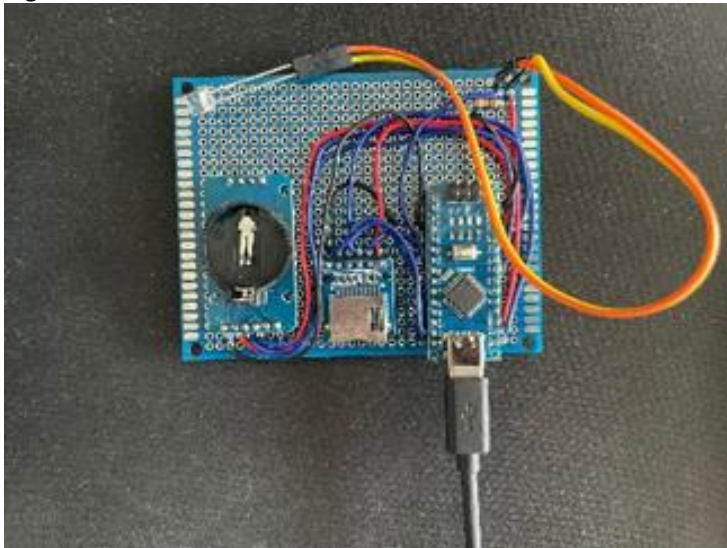
Figuur 3 - Schema sensor prototype in KiCad

Belangrijkste aansluitingen:

- Fotodiode → analoge ingang A3 van de Arduino Nano, met een 10kΩ pull-down weerstand naar GND.
- RTC DS3231 → I2C-bus (SDA op A4, SCL op A5).
- SD-kaartmodule → SPI-bus (MOSI op D11, MISO op D12, SCK op D13, CS op D10).
- Voeding → 5V en GND van de powerbank via USB naar de Arduino, die de overige modules voedt.

4.3 Solderen op protoboard

Na goedkeuring van het schema zijn de componenten gesoldeerd op een soldeerboardje (protoboard). Hierbij is gebruikgemaakt van solid core kabel (0,25 mm) voor vaste verbindingen. Tijdens het solderen deed zich een complicatie voor: twee RTC DS3231-modules bleken defect te zijn na het solderen. Dit heeft ertoe geleid dat de klokmodule twee keer opnieuw gesoldeerd en vervangen moest worden voordat het systeem correct functioneerde. Dit benadrukt het belang van het testen van componenten vóór definitieve montage. Het gesoldeerde protoboard is te zien in Figuur 4



Figuur 4 - Sensor schakling op protoboard.

5. Behuizing

Voor de behuizing van het sensorsysteem is een modulaire ontwerp gekozen. Zowel de zender (laser) als de ontvanger (fotodiode met protoboard) bestaan elk uit vijf onderdelen, waarvan drie identiek zijn. Dit betekent dat deze drie onderdelen onderling verwisselbaar zijn, wat het systeem eenvoudiger maakt om te produceren en te onderhouden.

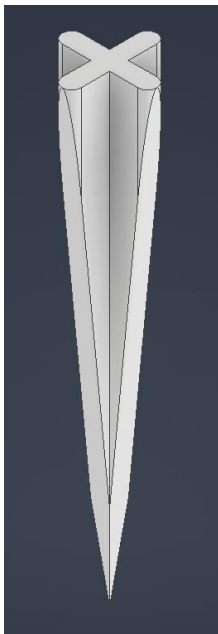
De gedeelde onderdelen zijn de prikker, het tussenstuk en de sensorhouder. De behuizing zelf verschilt per module omdat de ruimte-inwendige eisen anders zijn. Een overzicht van alle onderdelen van de behuizing is te zien in Figuur 5.



Figuur 5 - Overzicht van alle onderdelen van de behuizing

Onderdeel 1 - De prikker

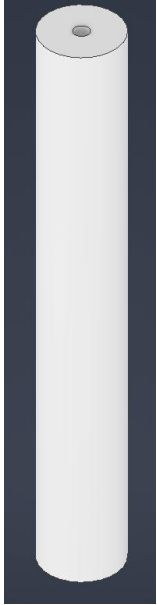
De prikker is een spits toelopend onderdeel dat in de grond wordt gestoken, vergelijkbaar met de pennen van buitenlampen zo als te zien is in Figuur 6. Dit zorgt voor een stabiele en rechte plaatsing in de buitenomgeving zonder dat er geboord of geschroefd hoeft te worden in bestaande constructies.



Figuur 6 - De prikker

Onderdeel 2 - Het tussenstuk

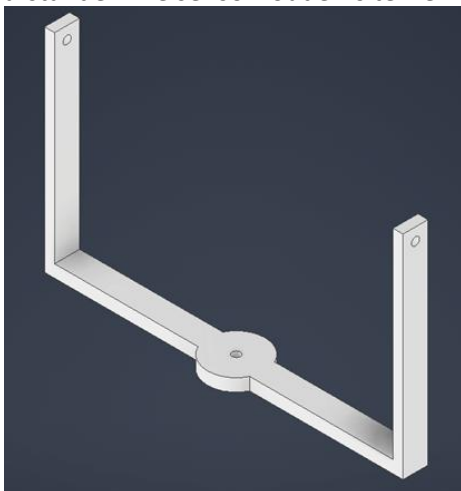
Het tussenstuk is cilindrisch van vorm en verbindt de prikker met de rest van de opstelling zo als te zien is in Figuur 7. Het zorgt ervoor dat de zender en ontvanger op de juiste hoogte komen te staan om ook auto's betrouwbaar te kunnen detecteren. Aan de onderkant past de prikker in het tussenstuk. Aan de bovenkant zit een gat voor een heat insert, zodat er een schroefdraadverbinding ontstaat waarmee de sensorhouder horizontaal verstelbaar is.



Figuur 7 - Het tussenstuk

Onderdeel 3 - De sensorhouder

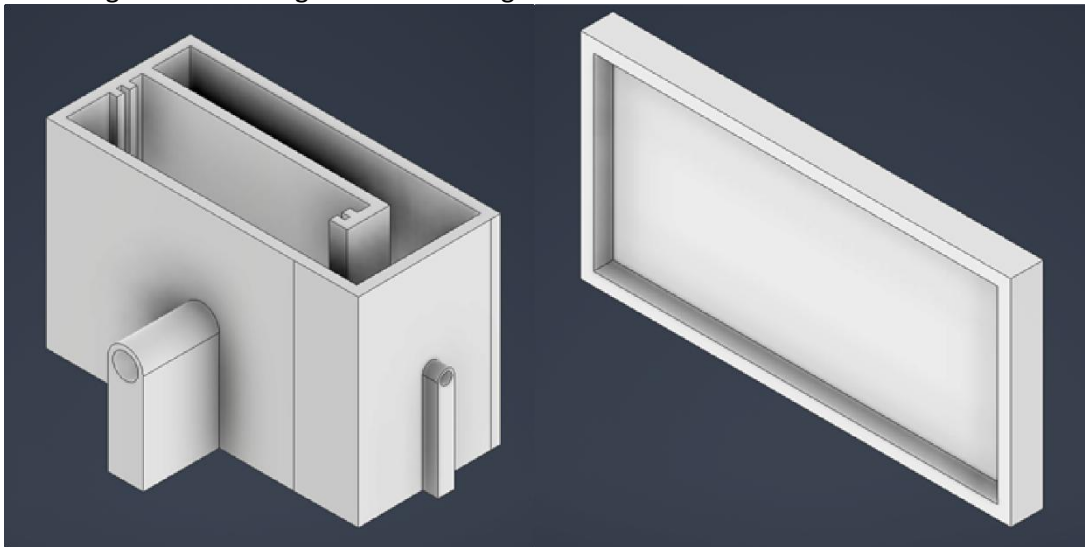
De sensorhouder verbindt het tussenstuk met de behuizing van de sensor. Hij heeft drie gaten: twee voor de bevestiging aan het tussenstuk en één om de behuizing van de sensor zelf te kunnen kantelen. Door deze kantelverbinding kunnen zowel de zender als de ontvanger nauwkeurig op elkaar worden gericht, wat essentieel is voor een betrouwbare laserverbinding over langere afstanden. De sensorhouder is te zien in Figuur 8.



Figuur 8 - De sensorhouder

Onderdeel 4-5a - Behuizing ontvanger (fotodiode + protoboard)

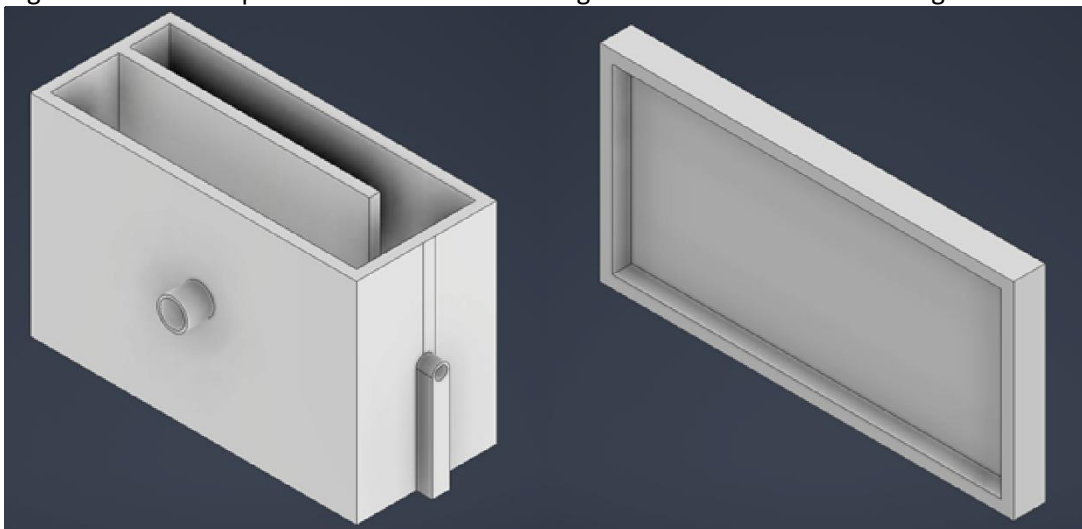
De behuizing van de ontvanger is de grootste van de twee sensorbehuizingen. Aan de zijkant zitten twee gaten met heat inserts voor bevestiging aan de sensorhouder. De voorkant heeft een verlengde loop, vergelijkbaar met een kokertje, om direct invallend zonlicht zoveel mogelijk te weren. Dit vermindert valse detecties door omgevingslicht. Binnenin is ruimte voor de powerbank, het protoboard en de bedrading. Aan de onderkant zit een uitsparing waar een schakelaar in past om het systeem van stroom te voorzien. De behuizing wordt afgesloten met een passend deksel. De behuizing van de ontvanger is te zien in Figuur 9.



Figuur 9 - Behuizing ontvanger

Onderdeel 4-5b - Behuizing zender (laser)

De behuizing van de zender lijkt sterk op die van de ontvanger en heeft dezelfde bevestigingsgaten met heat inserts aan de zijkant. Omdat er alleen een lasermodule, een powerbank en een schakelaar in zitten, is deze behuizing compacter dan die van de ontvanger. Ook deze behuizing wordt afgesloten met een passend deksel. De behuizing van de zender is te zien in Figuur 10.



Figuur 10 - Behuizing zender

Ontwerpiteraties

Tijdens het 3D-printen van de behuizing zijn twee onderdelen aangepast op basis van praktische problemen die ontstonden bij de eerste printpoging.

Het tussenstuk (onderdeel 2) was in het oorspronkelijke ontwerp cilindrisch van vorm. De printer had moeite met het correct printen van deze ronde geometrie, waardoor het onderdeel niet bruikbaar uit de printer kwam. Als oplossing is de vorm aangepast van cilindrisch naar vierkant. Dit maakte het onderdeel beter printbaar zonder dat de functionaliteit verloren ging.

De sensorhouder (onderdeel 3) bleek na de eerste print te dun en daardoor niet sterk genoeg om de behuizing stabiel te houden. Om dit op te lossen is de wanddikte vergroot, waardoor het onderdeel voldoende stevigheid kreeg voor gebruik in de buitenomgeving.

De definitieve versie van alle onderdelen na deze aanpassingen is te zien in Figuur 11.



Figuur 11 - Definitieve onderdelen na ontwerpiteraties

Printoverzicht behuizing

Alle onderdelen zijn geprint met PLA-filament op een Ender V3 SE. Onderstaande Tabel 3 geeft een overzicht van het materiaalverbruik en de printtijd per onderdeel.

Tabel 3 - Printoverzicht behuizing

Onderdeel	Aantal	Materiaal per stuk (PLA)	Printtijd per stuk	Printer
Onderdeel 1 - De prikker	2x	8 gram	1:31 uur	Ender V3 SE
Onderdeel 2 - Het tussenstuk	2x	52 gram	3:00 uur	Ender V3 SE
Onderdeel 3 - De sensorhouder	2x	45 gram	4:03 uur	Ender V3 SE
Onderdeel 4a - Behuizing ontvanger	1x	153 gram	14:47 uur	Ender V3 SE
Onderdeel 4b - Behuizing zender	1x	140 gram	13:19 uur	Ender V3 SE
Onderdeel 5a - Deksel ontvanger	1x	36 gram	2:05 uur	Ender V3 SE
Onderdeel 5b - Deksel zender	1x	34 gram	1:58 uur	Ender V3 SE
Totaal	10 prints	573 gram	±49:17 uur	-

Resultaat

Na het printen en assembleren van alle onderdelen is het complete sensorsysteem tot stand gekomen. Onderstaande afbeelding Figuur 12 tonen het eindresultaat van zowel de zender als de ontvanger in de praktijk.



Figuur 12 - Complete opstelling zender en ontvanger

6. Ontwikkeling van de code

6.1 Losse component tests

Voordat de volledige code werd geschreven, zijn alle drie de modules eerst afzonderlijk getest in aparte Arduino-sketches. Voor de fotodiode is getest of de analoge waarden correct binnenkwamen via de Serial Monitor. De RTC DS3231 is getest op correcte tijdregistratie en de SD-kaartmodule is getest op het aanmaken en wegschrijven van een testbestand. Door componenten los te testen konden fouten per module worden opgespoord zonder dat andere onderdelen storing veroorzaakten.

6.2 Sensor Code V1 - eerste integratie

Na de losse tests zijn alle modules samengevoegd tot één werkende code: Sensor Code V1. De belangrijkste keuzes in V1 waren:

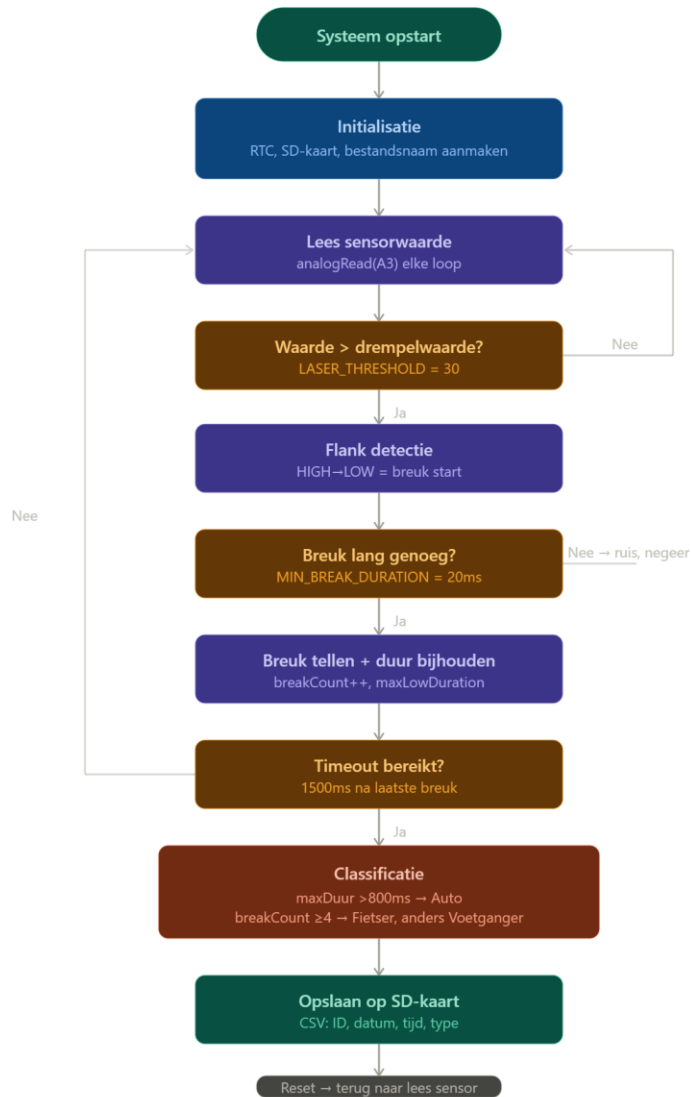
- De fotodiode was aangesloten op digitale pin D2 en werd uitgelezen met `digitalRead()`.
- Het logbestand had een vaste naam: LOG.CSV.
- Detectie was gebaseerd op drie patronen: een lange onderbreking voor een auto, meerdere snelle onderbrekingen voor een fietser en een korte onderbreking voor een voetganger.

6.3 Sensor Code V2 - verbeteringen

Tijdens het testen van V1 buiten bleek dat de fotodiode te gevoelig was voor zonlicht, waardoor het systeem continu een laser registreerde terwijl die er niet was. Dit leidde tot de volgende verbeteringen in V2:

- Overstap van `digitalRead()` op `analogRead()` op pin A3, zodat een instelbare drempelwaarde (LASER_THRESHOLD) bepaalt wanneer de laser als aanwezig wordt beschouwd.
- Toevoeging van MIN_BREAK_DURATION om ruis en trillingen te filteren: onderbrekingen korter dan 20ms worden genegeerd.
- Het logbestand krijgt automatisch een naam op basis van datum en tijd van inschakelen, zodat meerdere meetmomenten te onderscheiden zijn.
- De sensorwaarde wordt nu ook geprint in de Serial Monitor voor eenvoudiger debuggen.

De werking van V2 is schematisch weergegeven in Figuur 13.



Figuur 13 - Flowchart sensor code v2

5.4 Sensor Code V3 - verbeterde code volgens richtlijnen

Na het ontwikkelen van V2 is de code herschreven naar V3 volgens de C-coding richtlijnen van de opleiding. De belangrijkste verbeteringen zijn:

- Alle constanten staan als #define bovenaan in hoofdletters, zodat direct zichtbaar is dat het om instellingen gaat.
- Elke functie heeft een documentatieblok met beschrijving, argumenten en returnwaarde.
- Bovenaan het bestand staat een volledige bestandsdocumentatie met naam, auteur, versie en beschrijving.
- De loop() functie bevat alleen nog functieaanroepen. Alle logica zit in aparte, herbruikbare functies.
- Globale variabelen beginnen met een g zodat direct zichtbaar is dat het globale variabelen zijn.
- Functieprototypes staan bovenaan zodat de volgorde van functies vrij is.
- Accolades staan lijnrecht onder elkaar met 4 spaties inspringing.

De code is opgesplitst in de volgende functies in Tabel 4:

Tabel 4 – Functies code

Functie	Beschrijving
initRTC()	Initialiseert de klokmodule
initSD()	Initialiseert de SD-kaartmodule
initLogFile()	Maakt logbestand aan met datum/tijd naam
readSensor()	Leest fotodiode en vergelijkt met drempelwaarde
handleBreakStart()	Verwerkt start van een laseronderbreking
handleBreakEnd()	Verwerkt einde van een onderbreking, filtert ruis
updateMaxDuration()	Houdt maximale breukduur bij
checkTimeout()	Controleert of evaluatiemoment bereikt is
classifyObject()	Classificeert passant als voetganger, fietser of auto
logData()	Schrijft detectie weg naar CSV op SD-kaart
resetMeting()	Reset alle variabelen voor volgende passage

De volledige code is te zien hieronder:

```
/*
 *
 * =====
 ==
 * Sensor_code_V3.ino
 * Auteur      : Thijs Buisman
 * Versie      : 3.0
 * Datum       : 2026
 * -----
 *
 * Beschrijving:
 *   Geautomatiseerd passantenmeetsysteem voor de buitenruimte van wijk
 *   Cartesius. Het systeem detecteert onderbrekingen van een laserstraal
 *   via een fotodiode en classificeert passanten als voetganger, fietser
 *   of auto op basis van onderbrekingspatronen. Data wordt met tijdstempel
 *   weggeschreven naar een CSV-bestand op een SD-kaart.
 *
 * Hardware:
 *   - Arduino Nano
 *   - Fotodiode op pin A3 met 10kOhm pull-down weerstand
 *   - RTC DS3231 op I2C (SDA=A4, SCL=A5)
 *   - SD-kaartmodule op SPI (MOSI=D11, MISO=D12, SKC=D13, CS=D4)
 *
 * Versiebeheer:
 *   V1 - Eerste werkende integratie, digitalRead op D2
 *   V2 - analogRead op A3, drempelwaarde, dynamische bestandsnaam
 *   V3 - Modulaire opbouw, C-coding richtlijnen, functieprototypes
```

```

*
=====
==
*/

#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include <RTClib.h>

/* --- Pin definities --- */
#define SENSOR_PIN      A3
#define SD_CS_PIN       4

/* --- Sensor drempelwaarden --- */
#define LASER_THRESHOLD  25    /* Analoge waarde waarboven laser
aanwezig is */
#define MIN_BREAK_DURATION 20    /* Minimale breukduur in ms, korter =
ruis */

/* --- Detectie instellingen --- */
#define CAR_MIN_DURATION  800    /* Breukduur > 800ms =
auto */
#define BIKE_MIN_INTERRUPTS 4    /* 4 of meer breuken =
fietser */
#define TIMEOUT_MS        1500  /* Wachtijd na laatste breuk voor
evaluatie */

/* --- Overige instellingen --- */
#define SENSOR_ID         "0x001b"
#define CSV_HEADER        "SensorID;Datum;Tijd;GedetecteerdObject"

/* --- Globale variabelen --- */
RTC_DS3231    rtc;
char          gLogFile[13];
bool          gIsMeasuring;
bool          gLastSensorState;
unsigned long gCurrentLowStart;
unsigned long gMaxLowDuration;
unsigned long gLastInterruptTime;
int           gBreakCount;

/* --- Functieprototypes --- */
void  initRTC(void);
void  initSD(void);
void  initLogFile(void);
bool  readSensor(void);
void  handleBreakStart(unsigned long currentTime);

```

```

void    handleBreakEnd(unsigned long currentTime);
void    updateMaxDuration(unsigned long currentTime);
bool    checkTimeout(unsigned long currentTime);
String  classifyObject(void);
void    logData(String type);
void    resetMeting(void);

/*
 * setup()
 * Initialiseert alle hardware modules bij opstarten.
 * Wordt eenmalig uitgevoerd na inschakelen of reset.
 */
void setup()
{
    Serial.begin(9600);
    gIsMeasuring      = false;
    gLastSensorState  = HIGH;
    gCurrentLowStart   = 0;
    gMaxLowDuration    = 0;
    gLastInterruptTime = 0;
    gBreakCount        = 0;
    initRTC();
    initSD();
    initLogFile();
    Serial.println("Systeem klaar voor detectie.");
}

/*
 * loop()
 * Hoofdlus: leest sensor, verwerkt flanken en evalueert na timeout.
 */
void loop()
{
    bool          currentSensorState = readSensor();
    unsigned long currentTime         = millis();

    if (gLastSensorState == HIGH && currentSensorState == LOW)
        handleBreakStart(currentTime);

    if (gLastSensorState == LOW && currentSensorState == HIGH)
        handleBreakEnd(currentTime);

    if (gIsMeasuring && currentSensorState == LOW)
        updateMaxDuration(currentTime);

    if (gIsMeasuring && currentSensorState == HIGH &&
checkTimeout(currentTime))
    {

```



```

        String objectType = classifyObject();
        logData(objectType);
        resetMeting();
    }

    gLastSensorState = currentSensorState;
}

/*
 * initRTC()
 * Initialiseert de RTC DS3231 klokmodule via I2C.
 * Bij stroomverlies wordt tijd ingesteld op compileer-tijdstip.
 */
void initRTC(void)
{
    if (!rtc.begin())
    {
        Serial.println("FOUT: RTC niet gevonden!");
        while (1);
    }
    if (rtc.lostPower())
    {
        Serial.println("RTC: stroom verloren, tijd hersteld.");
        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    }
}

/*
 * initSD()
 * Initialiseert de SD-kaartmodule via SPI.
 * Stopt het programma als de module niet gevonden wordt.
 */
void initSD(void)
{
    Serial.print("SD-kaart initialiseren...");
    if (!SD.begin(SD_CS_PIN))
    {
        Serial.println(" MISLUKT!");
        while (1);
    }
    Serial.println(" gelukt.");
}

/*
 * initLogFile()
 * Maakt logbestand aan met naam op basis van datum en tijd.
 * Bestandsnaam formaat: MMDDHHII.CSV (FAT32 max 8.3)
 */

```

```

void initLogFile(void)
{
    DateTime now = rtc.now();
    sprintf(gLogFile, "%02d%02d%02d%02d.CSV",
        now.month(), now.day(), now.hour(), now.minute());
    Serial.print("Logbestand: ");
    Serial.println(gLogFile);
    if (!SD.exists(gLogFile))
    {
        File logFile = SD.open(gLogFile, FILE_WRITE);
        if (logFile)
        {
            logFile.println(CSV_HEADER);
            logFile.close();
        }
    }
}

/*
 * readSensor()
 * Leest analoge waarde fotodiode, vergelijkt met drempelwaarde.
 * Returnvalue: TRUE als laser aanwezig, FALSE als onderbroken
 */
bool readSensor(void)
{
    int waarde = analogRead(SENSOR_PIN);
    return (waarde > LASER_THRESHOLD) ? HIGH : LOW;
}

/*
 * handleBreakStart()
 * Verwerkt het moment waarop de laserstraal wordt onderbroken.
 * Arguments: currentTime - huidige millis() waarde
 */
void handleBreakStart(unsigned long currentTime)
{
    if (!gIsMeasuring)
    {
        gIsMeasuring    = true;
        gBreakCount      = 0;
        gMaxLowDuration = 0;
    }
    gBreakCount++;
    gCurrentLowStart    = currentTime;
    gLastInterruptTime = currentTime;
}

/*

```

```

    * handleBreakEnd()
    * Verwerkt het moment waarop de laserstraal herstelt.
    * Breuken korter dan MIN_BREAK_DURATION worden als ruis genegeerd.
    * Arguments: currentTime - huidige millis() waarde
    */
void handleBreakEnd(unsigned long currentTime)
{
    if (!gIsMeasuring)
        return;

    unsigned long lowDuration = currentTime - gCurrentLowStart;

    if (lowDuration >= MIN_BREAK_DURATION)
    {
        if (lowDuration > gMaxLowDuration)
            gMaxLowDuration = lowDuration;
        gLastInterruptTime = currentTime;
    }
    else
        gBreakCount--;
}

/*
 * updateMaxDuration()
 * Houdt maximale breukduur bij tijdens actieve onderbreking.
 * Arguments: currentTime - huidige millis() waarde
 */
void updateMaxDuration(unsigned long currentTime)
{
    unsigned long currentDuration = currentTime - gCurrentLowStart;
    if (currentDuration > gMaxLowDuration)
        gMaxLowDuration = currentDuration;
}

/*
 * checkTimeout()
 * Controleert of wachttijd na laatste breuk verstreken is.
 * Returnvalue: TRUE als timeout bereikt, anders FALSE
 * Arguments: currentTime - huidige millis() waarde
 */
bool checkTimeout(unsigned long currentTime)
{
    return (currentTime - gLastInterruptTime > TIMEOUT_MS);
}

/*
 * classifyObject()
 * Classificeert gedetecteerd object op basis van breukpatroon.

```

```

    * Returnvalue: String "Auto", "Fietser" of "Voetganger"
    */
String classifyObject(void)
{
    String objectType;

    if (gMaxLowDuration > CAR_MIN_DURATION)
        objectType = "Auto";
    else if (gBreakCount >= BIKE_MIN_INTERRUPTS)
        objectType = "Fietser";
    else
        objectType = "Voetganger";

    Serial.print("Detectie: ");    Serial.print(objectType);
    Serial.print(" | Breuken: "); Serial.print(gBreakCount);
    Serial.print(" | Max duur: "); Serial.print(gMaxLowDuration);
    Serial.println("ms");

    return objectType;
}

/*
 * logData()
 * Schrijft detectieregel naar CSV-logbestand op SD-kaart.
 * Formaat: SensorID;Datum;Tijd;GedetecteerdObject
 * Arguments: type - gedetecteerd objecttype als String
 */
void logData(String type)
{
    DateTime now = rtc.now();
    char dateBuffer[12];
    char timeBuffer[10];

    sprintf(dateBuffer, "%04d-%02d-%02d", now.year(), now.month(),
now.day());
    sprintf(timeBuffer, "%02d:%02d:%02d", now.hour(), now.minute(),
now.second());

    String logString = String(SENSOR_ID) + ";" + dateBuffer + ";" +
timeBuffer + ";" + type;

    File logFile = SD.open(gLogFile, FILE_WRITE);
    if (logFile)
    {
        logFile.println(logString);
        logFile.close();
        Serial.println("Opgeslagen in: " + String(gLogFile));
    }
}

```

```
        else
            Serial.println("FOUT: kan bestand niet openen: " +
String(gLogFile));
    }

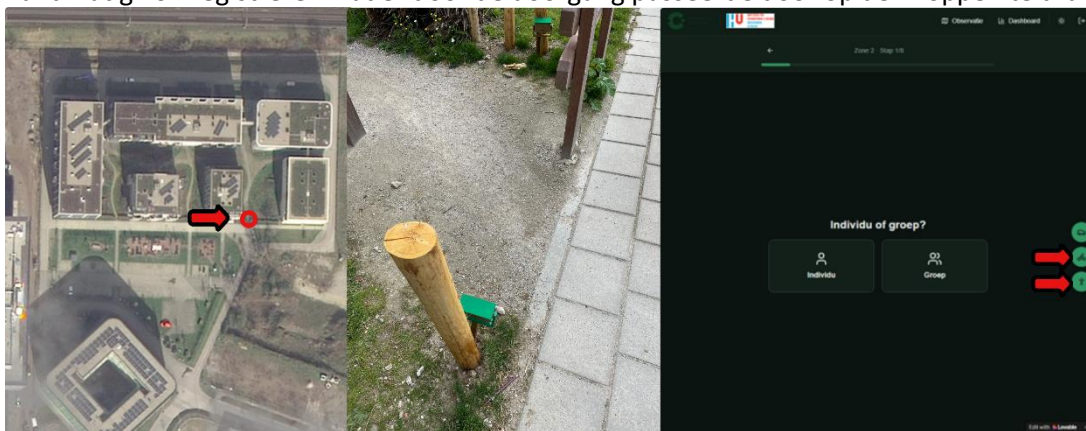
    /*
     * resetMeting()
     * Reset alle metvariabelen voor de volgende passage.
     */
    void resetMeting(void)
    {
        gIsMeasuring = false;
    }
}
```

7. Testen & resultaten

7.1 Testopstelling

Op 21 mei 2026 is het sensorprototype gedurende vier uur getest in de buitenruimte van Cartesius bij de doorgang van deelgebied 2. De test vond plaats tussen 14:45 en 18:45 onder bewolkte en droge weersomstandigheden. De afstand tussen de laser en de fotodiode bedroeg circa 2 meter. Tijdens de test is één observator ter plaatse gebleven om tegelijkertijd handmatig te registreren wat er door de doorgang passeerde via de SOPARC-webapp. Op die manier kon de sensordata achteraf worden vergeleken met de handmatige tellingen.

In Figuur 14 is de testopstelling weergegeven. Links is op de satellietfoto te zien waar de sensor is neergezet bij de doorgang van deelgebied 2. In het midden is de fysieke opstelling van de sensor op locatie te zien. Rechts is de SOPARC-webapp te zien waarmee de observator tijdens de test handmatig kon registreren wat er door de doorgang passeerde door op de knoppen te drukken.



Figuur 14 - Testopstelling: locatie (links), sensoropstelling op locatie (midden) en SOPARC-webapp voor validatie (rechts)

7.2 Resultaten

Om de nauwkeurigheid van het prototype te kunnen beoordelen, zijn de CSV-data van de sensor en de handmatige registraties uit de SOPARC-webapp naast elkaar gelegd in een Excel-bestand. Beide datasets bevatten per passage een tijdstempel en een gebruikerstype, waardoor de registraties op volgorde vergeleken konden worden. Een screenshot van dit overzicht is te zien in Figuur 15.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	SensorID	Datum	Tijd	GedetecteerdObject			id	button	clicked_at	zone_id			
2	0x001b	20-5-2026	13:45:18	Voetganger			567	Voetgang	20-5-2026 13:45	2			
3	0x001b	20-5-2026	13:47:35	Voetganger			f44	Voetgang	20-5-2026 13:47	2			
4	0x001b	20-5-2026	13:49:24	Voetganger			cfa	Voetgang	20-5-2026 13:49	2			
5	0x001b	20-5-2026	13:51:56	Fietser			70f	Voetgang	20-5-2026 13:51	2			
6	0x001b	20-5-2026	13:58:32	Fietser			c54	Fietser	20-5-2026 13:58	2			
7	0x001b	20-5-2026	14:01:52	Voetganger			5e1	Fietser	20-5-2026 14:01	2			
8	0x001b	20-5-2026	14:05:22	Voetganger			19f	Voetgang	20-5-2026 14:05	2			
9	0x001b	20-5-2026	14:09:41	Fietser			8e1	Fietser	20-5-2026 14:09	2			
10	0x001b	20-5-2026	14:12:32	Voetganger			e9f	Voetgang	20-5-2026 14:12	2			
11	0x001b	20-5-2026	14:20:54	Fietser			7f6	Fietser	20-5-2026 14:20	2			
12	0x001b	20-5-2026	14:22:13	Voetganger			87e	Voetgang	20-5-2026 14:22	2			
13	0x001b	20-5-2026	14:30:24	Fietser			b1f	Voetgang	20-5-2026 14:30	2			
14	0x001b	20-5-2026	14:32:28	Voetganger			d5f	Voetgang	20-5-2026 14:32	2			
15	0x001b	20-5-2026	14:35:45	Voetganger			19e	Voetgang	20-5-2026 14:35	2			
16	0x001b	20-5-2026	14:39:23	Fietser			a8f	Fietser	20-5-2026 14:39	2			
17	0x001b	20-5-2026	14:41:13	Fietser			7df	Fietser	20-5-2026 14:41	2			

Figuur 15 - Screenshot van de samengevoegde sensor- en webapp-data in Excel

Uit deze vergelijking kwamen de volgende resultaten naar voren. De sensor registreerde in totaal 81 passages, de handmatige telling via de webapp telde 79 passages. De resultaten per gebruikerstype zijn weergegeven in

Tabel 5 en

Tabel 6.

Tabel 5 - Sensordata: geregistreeerde passages

Gebruikerstype	Aantal
Voetganger	50
Fietser	28
Auto	3
Totaal	81

Tabel 6 - Handmatige telling via webapp: geregistreeerde passages

Gebruikerstype	Aantal
Voetganger	47
Fietser	32
Auto	0
Totaal	79

7.3 Vergelijking en nauwkeurigheid

Door de twee datasets naast elkaar te leggen, kan per gebruikerstype worden bepaald hoe nauwkeurig de sensor heeft gepresteerd. Dit is weergegeven in Tabel 7.

Tabel 7 - Vergelijking sensor vs. handmatige telling

Gebruikerstype	Sensor	Handmatig	Verschil	Afwijking
Voetganger	50	47	+3	6,4%
Fietser	28	32	-4	12,5%
Auto	3	0	+3	—
Totaal	81	79	+2	2,5%

Voor voetgangers en fietsers presteert de sensor redelijk goed. Het totale verschil van 2 passages op 79 geregistreeerde passages komt neer op een afwijking van 2,5%, wat binnen de gestelde eis van maximaal 15% foutmarge valt. Per gebruikerstype liggen de afwijkingen echter hoger: voor voetgangers bedraagt de afwijking 6,4% en voor fietsers 12,5%. De fietserdetectie zit daarmee dicht tegen de grens van 15% aan.

De drie geregistreeerde auto's verdienen extra toelichting. De handmatige observator heeft geen auto's genoteerd in de webapp, wat betekent dat de sensor deze passages fout heeft geclassificeerd. Mogelijk gaat het om fietsers of voetgangers die laser langdurig blokkeerden, waardoor de sensor dit interpreteerde als een auto. Dit is een bekend beperkingspunt van het huidige classificatiealgoritme dat uitsluitend kijkt naar de duur van de onderbreking.

De vier gemiste fietsers aan de kant van de sensor kunnen verklaard worden doordat niet alle fietsen dezelfde spaakpatronen hebben. Fietsen met weinig of dicht op elkaar staande spaken genereren minder onderbrekingen, waardoor de sensor deze soms classificeert als voetganger in plaats van fietser.

7.4 Toetsing aan systeemeisen

Op basis van de testresultaten kan worden beoordeeld in hoeverre het prototype voldoet aan de vooraf opgestelde systeemeisen. Dit is weergegeven in Tabel 8.

Tabel 8 - Toetsing systeemeisen

MoSCoW	Eis	Resultaat	Toelichting
Must have	Betrouwbare detectie bij minimaal 2 meter afstand	✓	De sensor functioneerde correct op een afstand van 1 tot 2 meter.
Must have	Elke detectie opgeslagen met correcte tijdstempel op SD-kaart	✓	Alle 81 passages zijn correct weggeschreven met datum en tijd.
Must have	Onderscheid tussen minimaal drie gebruikerstypen	✓	Het systeem classificeert voetgangers, fietsers en auto's.
Must have	Minimaal 8 uur werken op powerbank van 5000 mAh	✓	Het systeem heeft de volledige testduur van 4 uur zonder onderbreking gefunctioneerd. De berekende batterijduur bedraagt circa 40 uur.
Should have	Betrouwbaar werken bij direct zonlicht zonder valse detecties	✓	De test vond plaats bij bewolkt weer. Bij eerdere tests buiten zonder kokertje traden wel valse detecties op door zonlicht. Dit is opgelost met de analoge drempelwaarde.
Should have	Foutmarge maximaal 15% ten opzichte van handmatige telling	✓	De totale afwijking bedraagt 2,5%. Per type: voetganger 6,4%, fietser 12,5%. Beide vallen binnen de gestelde grens.
Should have	Geen dubbeltellingen bij één passage	✗	Door de hold-off tijd en minimale breukduur zijn geen dubbeltellingen waargenomen. Toch zijn er 2 extra meting gemten.
Could have	Detectie van looprichting	✗	Niet geïmplementeerd in dit prototype. Vereist een tweede sensormodule.
Could have	Automatische export naar leesbaar CSV met kolomkoppen	✓	Het systeem maakt automatisch een CSV-bestand aan met kolomkoppen en een bestandsnaam op basis van datum en tijd.
Won't have	Draadloze datatransmissie	—	Buiten scope voor dit prototype.
Won't have	Realtime monitoring op afstand	—	Buiten scope voor dit prototype.
Won't have	Automatische classificatie via machine learning	—	Buiten scope voor dit prototype.

Alle must have eisen zijn behaald. Van de drie should have eisen zijn er twee behaald, waarvan de fietser detectie met 12,5% dicht tegen de gestelde grens van 15% aanligt. De enige could have eis die niet is gerealiseerd is de richting detectie, waarvoor een tweede sensormodule nodig is.

8. Conclusie

Het sensorprototype heeft tijdens de test op 21 mei 2026 aangetoond dat geautomatiseerde passantentelling met een lasersysteem in de buitenruimte van Cartesius haalbaar is. Alle must have eisen zijn behaald en de totale foutmarge van 2,5% valt ruim binnen de gestelde grens van 15%. Het systeem heeft gedurende de volledige testperiode van vier uur zelfstandig gefunctioneerd zonder storingen of dataverlies.

Toch zijn er ook beperkingen zichtbaar geworden. De fietserdetectie zit met een afwijking van 12,5% dicht tegen de grens aan, de sensor heeft drie passages fout geclassificeerd als auto, en de eis rondom dubbeltellingen is niet volledig behaald doordat het totale aantal registraties twee hoger uitvalt dan de handmatige telling. Dit betekent dat het systeem als proof of concept werkt, maar nog niet nauwkeurig genoeg is voor zelfstandige inzet zonder handmatige controle.

Of het systeem ook betrouwbaar werkt bij direct zonlicht, regen en harde wind kon in deze testopstelling niet worden vastgesteld omdat de test plaatsvond bij bewolkt en droog weer. Voor definitieve conclusies over de betrouwbaarheid in alle weersomstandigheden is aanvullend testen nodig.

Desondanks laat dit prototype zien dat een eenvoudig en kostenefficiënt lasersysteem in staat is om met beperkte middelen betrouwbare teldata te genereren. Het vormt daarmee een solide eerste stap richting een volwaardige sensor die in de toekomst, met de aanbevolen doorontwikkelingen, zelfstandig en continu ingezet kan worden voor het meten van het gebruik van de buitenruimte in Cartesius.

9. Aanbeveling

9.1 Beperkingen van het huidige prototype

Tijdens de test zijn een aantal beperkingen naar voren gekomen die de inzetbaarheid van het huidige prototype in de praktijk beperken. Deze zijn hieronder samengevat.

De behuizing is te zwaar en te groot, waardoor het systeem instabiel wordt bij harde wind en makkelijk kan omvallen. De prikker is van kunststof, waardoor die op harde ondergrond niet goed de grond in gaat zonder eerst een gat te maken met gereedschap. Het correct richten van de laser op de fotodiode kost veel tijd en is nauwkeurig werk, wat het plaatsen van de sensor omslachtig maakt. De maximale meetafstand bedraagt circa twee meter omdat de laserbundel daarna te veel uitspreidt en de fotodiode het signaal niet meer betrouwbaar oppikt. De behuizing is niet waterdicht, waardoor meten bij regen niet mogelijk is. Het systeem kan niet onderscheiden of er meerdere personen of fietsers naast elkaar door de doorgang lopen, wat leidt tot ondertelling bij drukte. Tot slot kunnen dieren zoals katten of honden ook door de laserstraal lopen en worden dan geregistreerd als passant, wat de data kan vertekenen.

9.2 Oplossingen en verbeteringen

Voor elk van de genoemde beperkingen zijn verbeteringen mogelijk die in een volgende versie van het prototype doorgevoerd kunnen worden.

De behuizing kan lichter en compacter worden gemaakt door over te stappen op een PCB in plaats van een protoboard, zie sectie 9.3. Een smallere behuizing vangt minder wind en is stabiel. De prikker kan worden vervangen door een metalen variant of de sensor kan worden bevestigd aan een bestaand hekwerk of paal, wat zowel stabiel als eenvoudiger te plaatsen is.

Het richten kan eenvoudiger worden gemaakt door een verstelbaar scharnierend mechanisme met vergrendeling toe te voegen aan de sensorhouder, zodat de laser in één beweging gericht en vergrendeld kan worden. Een indicatie-LED of buzzer die aangeeft wanneer de laser de fotodiode correct raakt zou dit proces verder vereenvoudigen.

De meetafstand kan worden vergroot door een gefocuste lasermodule te gebruiken of een lens toe te voegen voor de fotodiode, zodat de bundel minder uitspreidt over langere afstanden.

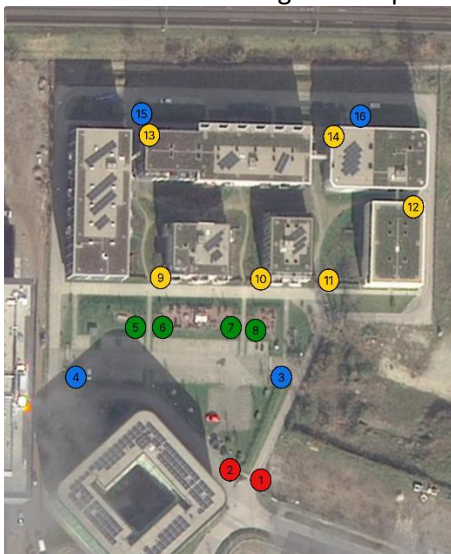
Voor waterdichtheid is een behuizing met minimaal IP65-certificering nodig. Dit betekent dat alle openingen zoals de schakelaarsparing en de lensopening worden afgedicht met rubberringen of een transparante afdekking.

De classificatie van gebruikerstypen kan worden verbeterd door een tweede sensormodule parallel aan de eerste te plaatsen. Hiermee kan de breedte van een object worden gemeten, wat helpt bij het onderscheiden van meerdere personen naast elkaar en het uitfilteren van kleine dieren.

9.3 Opschaling: van prototype naar volwaardig meetsysteem

Als de bovengenoemde verbeteringen zijn doorgevoerd, is het systeem geschikt voor bredere inzet in de wijk. De logische vervolgstap is het ontwerpen van een PCB op basis van het KiCad-schema dat voor dit prototype is gemaakt. Een PCB vervangt het protoboard door een compacte, op maat gemaakte printplaat waarop alle componenten direct gesoldeerd zijn. Dit maakt het systeem kleiner, robuuster en eenvoudiger te reproduceren in meerdere exemplaren. Een eerste versie van dit PCB-ontwerp is uitgewerkt in Bijlage G.

Met meerdere identieke sensoren is het mogelijk om op strategische plekken in de wijk te meten. Op basis van de plattegrond van Cartesius zijn in totaal 16 mogelijke meetlocaties geïdentificeerd, verdeeld over vier categorieën op basis van prioriteit en meetdoel. Dit is weergegeven in Figuur 16.



Figuur 16 - Aanbevolen sensorlocaties in Cartesius, gekleurd per categorie

Rood - hoofdingangen (locaties 1 en 2)

Locaties 1 en 2 bevinden zich bij de hoofdingangen van de wijk, waar op dit moment geen alternatieve route is. Al het verkeer dat de wijk binnenkomt of verlaat passeert langs deze punten, zowel voetgangers, fietsers als auto's. Dit maakt deze locaties het meest waardevol voor een totaalbeeld van het gebruik van de wijk. Met slechts twee sensoren kan hier al een volledig beeld worden verkregen van de totale in- en uitstroom.

Blauw - parkeerlocaties (locaties 3, 4, 15 en 16)

Locaties 3 en 4 zijn geplaatst bij het bestaande parkeerterrein en locaties 15 en 16 bij de parkeergarage. Deze locaties zijn relevant als het autogebruik specifiek in kaart gebracht moet worden. Voor algemeen onderzoek naar het gebruik van de buitenruimte zullen deze locaties minder prioriteit hebben, maar ze kunnen wel inzicht geven in hoe vaak bewoners de auto gebruiken ten opzichte van andere vervoersmiddelen.

Groen - moestuin en picknickzone (locaties 5 t/m 8)

Locaties 5 tot en met 8 zijn geplaatst rondom de moestuin en de picknicktafels. Door hier sensoren te plaatsen kan worden gemeten hoe vaak en op welke momenten deze voorzieningen worden bezocht. Dit geeft directe informatie over het gebruik van de recreatieve buitenruimte, wat aansluit bij de Blue Zone-principes die centraal staan in het onderzoek.

Geel - binnenhof deelgebied 2 (locaties 9 t/m 14)

Het binnenhof van deelgebied 2 heeft zes doorgangen. Door op elk van deze doorgangen een sensor te plaatsen kan het volledige gebruik van dit binnengebied in kaart worden gebracht. Omdat alle bewegingen via deze zes punten verlopen, is het met deze sensoren mogelijk om te bepalen hoeveel mensen het binnenhof gebruiken, op welke momenten het druk is en via welke doorgang de meeste mensen binnenkomen.

Door alle 16 locaties te combineren ontstaat een gedetailleerd en volledig beeld van hoe de buitenruimte van Cartesius wordt gebruikt, welke routes het meest worden belopen en op welke momenten de drukte het grootst is. Dit biedt een waardevolle aanvulling op de handmatige SOPARC-observaties en maakt het mogelijk om ook buiten de vaste meetmomenten data te verzamelen.

9.4 Overdracht aan een volgend groep

Dit prototype is een eerste stap. Voor een volgend groepje of een volgende onderzoeker dat verder wil bouwen op dit werk, is onderstaande overdracht bedoeld als startpunt.

Wat er al is: een werkend lasersysteem dat voetgangers, fietsers en auto's onderscheidt, data opslaat op een SD-kaart met tijdstempel, een 3D-geprinte behuizing op een verstelbare prikker, een gedocumenteerde Arduino-code (V3) die voldoet aan de C-coding richtlijnen en een eerste PCB-ontwerp in Bijlage G.

Wat er nog gedaan moet worden: een waterdichte behuizing ontwerpen van een weerbestendig materiaal zoals ASA of PETG in plaats van PLA, de prikker vervangen door een metalen variant of een wandbevestiging, het PCB-ontwerp laten produceren en testen, het classificatiealgoritme verbeteren voor betere fietserdetectie en het systeem testen bij direct zonlicht en regen om de betrouwbaarheid in alle weersomstandigheden te bevestigen.

Wat bruikbaar is als referentie: de systeemeisen in Tabel 18, de testresultaten in Tabel 24 en 25 en het KiCad-schema en de V3-code in Bijlage E vormen samen een gedocumenteerde basis waarop direct verder gebouwd kan worden.

]